부릉 외부 출금 API 사용 가이드

> 작성일: 2026-10-22
> 대상: 부릉 개발팀 (API 연동 담당자)

이 API가 하는 일

부릉 서버에서 Vpay를 통해 **직접 계좌 송금**을 요청할 수 있는 API입니다.

- 부릉이 요청 → Vpay 서버가 헥토 금융에 송금 처리
- 잔액이나 수수료 차감 없이 **전달받은 금액을 그대로 입금**
- 한 번 요청을 보내고, 결과를 주기적으로 확인하는 방식으로 동작

인증 (모든 요청에 필수)

모든 API 호출 시 아래 헤더를 반드시 포함해야 합니다.

```
X-API-Key: {Vpay로부터 전달받은 API Key}
```

> API Key는 별도로 안전한 경로(슬랙 DM 등)를 통해 전달받습니다. 코드나 공개된 곳에 노출하지 마세요.

1단계: 출금 요청 보내기

요청

```
POST https://back.vrplatformacct.com/api/v1/withdrawals
Content-Type: application/json
X-API-Key: {API Key}
```

```
{
  "requestId": "REQ-20261022-0001",
  "withdrawalId": "WD-20261022-0001",
  "transferAmount": "100000",
  "bankCode": "020",
  "accountNumber": "01012345678",
  "accountMemo": "부릉출금"
}
```

필드 설명

필드	필수	설명
`requestId`	✅	**이 API 호출의 고유 번호** — 재시도 시 동일 값 사용 (아래 참고)
`withdrawalId`	✅	**이 출금 건의 고유 번호** — 부릉 내부 출금 신청 번호
`transferAmount`	✅	송금할 금액 (원 단위, 숫자 문자열)
`bankCode`	✅	은행 코드 (예: 020=우리, 088=신한, 081=KEB하나)
`accountNumber`	✅	입금 계좌번호 (숫자만, 하이픈 없이)
`accountMemo`		입금자명 또는 적요 (통장에 표시되는 문구)

requestId와 withdrawalId 차이

	requestId	withdrawalId
의미	"이번 API 호출"의 번호	"이 출금 건"의 번호
재시도 시	**동일하게** 사용	**동일하게** 사용
예시	응답이 없어서 다시 보낼 때 → 동일 requestId 사용	이미 송금한 건을 다시 요청하면 오류 반환

> **핵심**: 응답을 못 받은 경우 동일한 `requestId`로 재요청해도 안전합니다. Vpay 서버가 중복 처리를 막아줍니다.

응답 예시 (202 Accepted)

```
{
  "requestId": "REQ-20261022-0001",
  "withdrawalId": "WD-20261022-0001",
  "transferId": "VT3F8A1B2C4D5E6F7A8B",
  "status": "ACCEPTED",
  "final": false,
  "transferAmount": "100000",
  "bankCode": "020",
  "maskedAccountNumber": "******5678",
  "currency": "KRW",
  "nextPollAfterSeconds": 5,
  "createdAt": "2026-10-22T12:00:00+09:00"
}
```

> 응답 코드가 **202**이면 접수 성공입니다. 아직 송금이 완료된 게 아니에요. `nextPollAfterSeconds` 초 후에 결과를 조회하면 됩니다.

2단계: 결과 조회하기

요청

```
GET https://back.vrplatformacct.com/api/v1/withdrawals/{requestId}
X-API-Key: {API Key}
```

응답 예시

```
{
  "requestId": "REQ-20261022-0001",
  "status": "SUCCEEDED",
  "final": true,
  "version": 3,
  "resultCode": "0000",
  "resultMessage": "정상처리",
  "completedAt": "2026-10-22T12:00:07+09:00"
}
```

상태(status) 설명

상태	의미	final	다음 행동
`ACCEPTED`	접수됨, 아직 송금 전	false	`nextPollAfterSeconds` (5초) 후 재조회
`PROCESSING`	헥토 금융에 송금 요청 중	false	`nextPollAfterSeconds` (10초) 후 재조회
`SUCCEEDED`	**송금 성공**	true	처리 완료
`FAILED`	송금 실패	true	실패 확인 후 재처리 여부 결정
`PENDING_CONFIRMATION`	결과 미확정 (수동 확인 필요)	false	Vpay 팀에 연락

폴링(주기적 조회) 권장 방법

1. POST 출금 요청 → 응답의 nextPollAfterSeconds 확인
2. 해당 초 후 GET 조회
3. final: true 이면 → 완료 (SUCCEEDED 또는 FAILED)
4. final: false 이면 → nextPollAfterSeconds 후 다시 조회
5. PENDING_CONFIRMATION 상태가 3회 연속이면 → 폴링 중단 후 Vpay 팀 문의

오류 응답

```
{
  "error": {
    "code": "IDEMPOTENCY_CONFLICT",
    "message": "동일한 requestId로 내용이 다른 요청이 존재합니다."
  }
}
```

오류 코드	HTTP	의미	해결 방법
`UNAUTHORIZED`	401	API Key가 없거나 틀림	API Key 확인
`VALIDATION_ERROR`	400	필수 필드 누락 또는 형식 오류	요청 데이터 확인
`IDEMPOTENCY_CONFLICT`	409	동일 requestId인데 금액 등 내용이 다름	요청 데이터 확인 (버그 가능성)
`WITHDRAWAL_ID_CONFLICT`	409	이미 처리된 출금 건을 새 requestId로 재요청	이중 송금 시도 — 처리 불가
`NOT_FOUND`	404	해당 requestId 없음	requestId 확인

PENDING_CONFIRMATION 상태가 지속될 때

헥토 금융에서 `VTIM(원천사 타임아웃)` 응답이 온 경우 실제 송금이 됐는지 확인이 필요한 상태입니다.

- Vpay 서버가 60초마다 자동으로 헥토에 결과를 재조회합니다
- 10회 재조회 후에도 결과가 나오지 않으면 → **알림 메일이 자동 발송됩니다**
- 이 경우 Vpay 팀에서 헥토에 직접 확인 후 수동 처리합니다
- 부릉 측에서는 `PENDING_CONFIRMATION`이 3회 연속으로 조회되면 폴링을 중단하고 Vpay 팀에 문의해주세요

은행 코드 전체 목록

코드	은행명
001	한국은행
002	산업은행
003	기업은행
004	국민은행
005	KEB 하나은행 (구 외환은행)
007	수협중앙회
008	수출입은행
011	농협은행

코드	은행명
012	지역농축협
020	우리은행
023	SC 제일은행
027	한국씨티은행
031	대구은행
032	부산은행
034	광주은행
035	제주은행
037	전북은행
039	경남은행
045	새마을금고중앙회
048	신협중앙회
050	상호저축은행
052	모건스탠리은행
054	HSBC 은행
055	도이치은행
071	우체국
081	KEB 하나은행
088	신한은행
089	케이뱅크
090	카카오뱅크
092	토스뱅크

구현 내용 정리 (Vpay 개발팀 참고용)

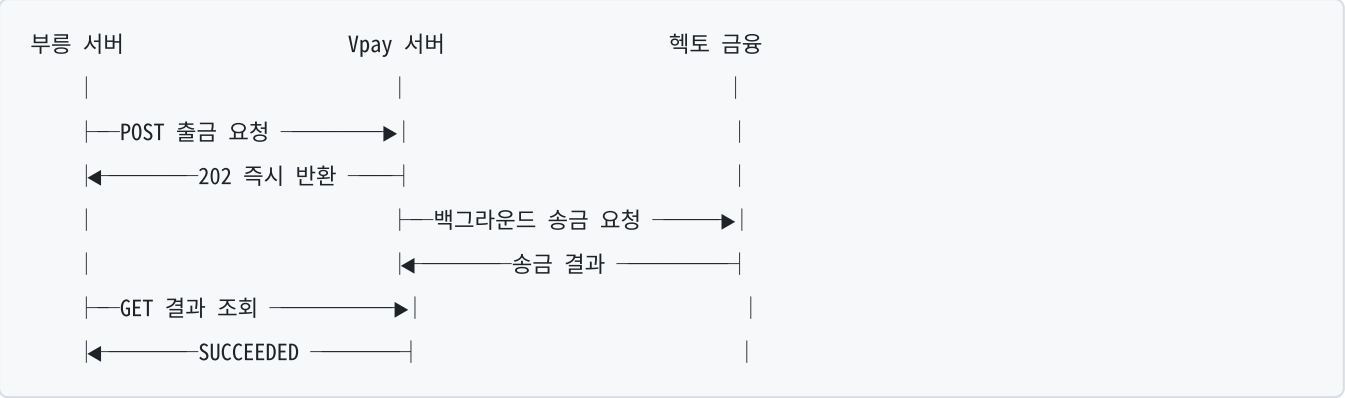
> 아래는 이 기능이 어떻게 만들어졌는지에 대한 정리입니다.

왜 이런 방식으로 만들었나요?

"요청만 받고 즉시 응답" 방식

헥토 금융 API는 응답까지 수 초가 걸릴 수 있습니다. 부릉 서버가 응답을 기다리다 타임아웃이 나면 재시도를 하게 되고, 이때 **이중 송금**이 발생할 수 있습니다.

그래서 Vpay 서버는 요청을 받으면 **즉시 202 응답**을 주고, 실제 송금은 백그라운드에서 처리합니다. 부릉 서버는 결과를 주기적으로 조회(폴링)하는 방식으로 최종 결과를 확인합니다.



이중 송금 방지

네트워크 문제로 같은 요청이 동시에 두 번 들어와도 한 번만 처리됩니다.

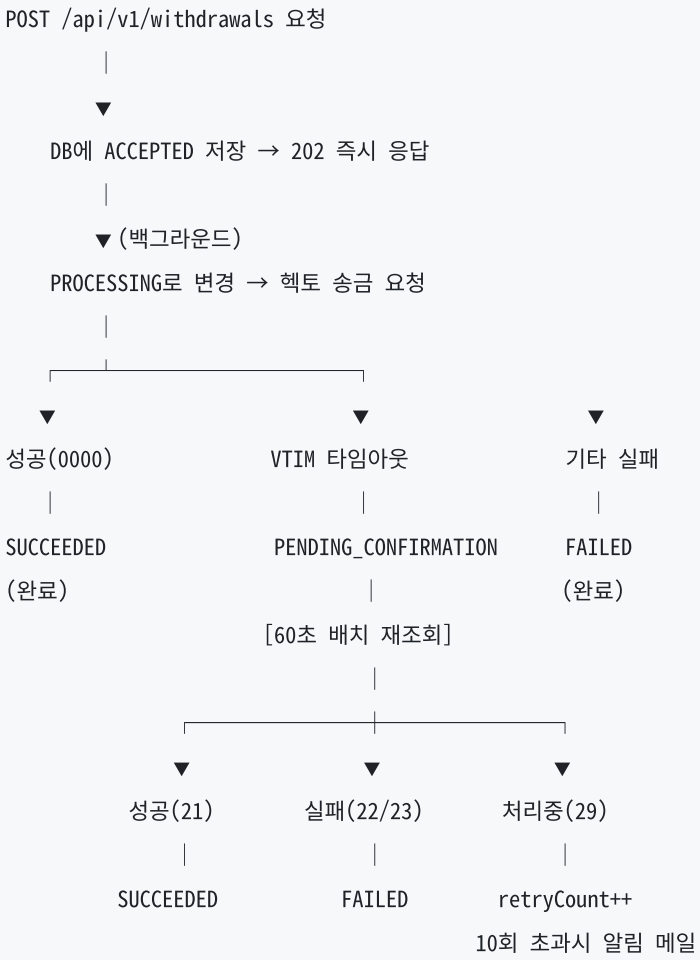
- `requestId` 단위로 DB에 잠금(Lock)을 걸어 두 번째 요청은 대기하게 함
- 잠금 해제 후 두 번째 요청이 들어오면 이미 저장된 결과를 그대로 반환
- 실제 헥토 호출은 단 한 번만 이루어짐

VTIM(원천사 타임아웃) 처리

헥토가 `VTIM` 응답을 보내는 경우, 실제 송금이 됐는지 안 됐는지 바로 알 수 없습니다.

- Vpay 서버는 이 상태를 `PENDING_CONFIRMATION`으로 저장
- 60초 간격 배치 스케줄러가 헥토에 결과를 재조회
- 결과가 나오면 SUCCEEDED 또는 FAILED로 자동 전환
- 10회(약 10분) 조회 후에도 결과 미확정이면 알림 메일 발송 → 수동 처리

주요 처리 흐름



계좌번호 보안 처리

계좌번호는 민감 정보이므로 아래와 같이 처리됩니다.

단계	처리 방식
DB 저장	AES-256 암호화하여 저장
헥토 전송	복호화 후 헥토 규격(AES-256-ECB)으로 재암호화
API 응답	뒤 4자리만 노출 (`maskedAccountNumber: "******1234"`)
로그	계좌번호 직접 출력 없음

배치 스케줄러 동작

`vroong-3pl-platform-batch` 서버에서 60초마다 아래 API를 호출합니다:

```
POST /api/v1/external/withdrawals/process-pending
```

이 배치가 `PROCESSING` 및 `PENDING_CONFIRMATION` 상태인 건들을 헥토에 재조회해서 결과를 업데이트합니다.

환경 설정 (운영 환경 적용 필요 항목)

설정	환경변수	설명
API Key	`VROONG_EXTERNAL_API_KEY`	부릉에 발급할 인증 키
알림 수신자	`vroong.notification.vtim.recipients`	VTIM 초과 시 알림 메일 수신 주소

배포 완료 현황 (2026-10-22 기준)

✓ DB 테이블 생성 완료

`vroong-backend` API 서버 배포 완료

`vroong-3pl-platform-batch` 배치 서버 배포 완료

✓ EC2 relay 서버 (`3.36.153.113`) `/relay/hecto/withdrawal-result` 엔드포인트 추가 완료

✓ End-to-End 테스트 완료